МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении курсовой работы

По дисциплине: «Функциональное программирование»

На тему: «Учет продуктивности работы сотрудников и расчет заработной платы на основе KPI»

Выполнил студент
группы 5130201/40002 _____ Храмов В. Д.

Проверил к. т. н. _____ Моторин Д. Е.

Санкт-Петербург
2026

# Содержание

Введение 3

1 Описание разработки проекта 4
1.1 Архитектура и модульная структура . . . . . 4
1.2 Технические особенности реализации . . . . . 4
1.3 Математическая модель . . . . . 5

2 Примеры работы программы 6

3 Руководство по эксплуатации для пользователя 10
3.1 Требования . . . . . 10
3.2 Порядок работы . . . . . 10

Заключение 11
Выгрузка коммитов по этапам . . . . . 11

Список литературы 14

# Введение

Предметная область работы — автоматизация учёта производственной деятельности сотрудников и расчёт заработной платы на основе KPI (Key Performance Indicators) [1, 2]. В прикладных информационных системах этот класс задач включает сбор первичных данных о задачах, их нормализацию, расчёт интегральных показателей эффективности, преобразование KPI в коэффициенты оплаты труда и формирование отчётности [3, 4].

В современном управлении KPI — инструмент количественной оценки результата труда, повышающий прозрачность мотивации [2, 5]. Важна не только формула KPI, но и её реализация: обработка неполных периодов, устойчивость к ошибкам, воспроизводимость расчётов и проверяемость логики тестами [4].

Для реализации выбрана парадигма функционального программирования и язык Haskell. Строгая типизация, чистые функции и мощные абстракции позволяют изолировать логику от ввода-вывода и упрощают тестирование [6, 7]. Практические аспекты работы с файлами и тестирования описаны в [8, 9].

# 1 Описание разработки проекта

## 1.1 Архитектура и модульная структура

Проект реализован в виде консольного приложения с разделением на логические модули:

- Core — доменные типы и общие структуры ошибок.
- DataLoader — загрузка CSV/JSON, парсинг и сериализация выходных отчетов.
- Productivity — вычисление индивидуальной и интегральной продуктивности.
- Salary — вычисление заработной платы с учетом KPI, бонусов и удержаний.
- Logging — логирование действий пользователя.
- Utils — вспомогательная валидация и работа с датами.
- app/Main.hs — консольный интерфейс и сценарий взаимодействия с пользователем.

## 1.2 Технические особенности реализации

Ключевые технические решения:

1. Строгая типизация предметной области. Для сущностей сотрудника, задачи, конфигурации и строк отчетов введены отдельные типы, что снижает риск логических ошибок.
2. Разделение доменной логики и I/O. Расчет KPI и зарплаты реализован чистыми функциями; операции чтения/записи изолированы в модуле загрузки.
3. Валидация входных данных. Реализована проверка формата месяца (ГГГГ-ММ), проверка существования месяца в загруженном наборе задач, обработка отсутствующих файлов и ошибок парсинга.
4. Учет отсутствий сотрудника. Добавлен отдельный входной файл absences.csv с полями больничных и неоплачиваемых дней по сотруднику и месяцу.
5. Тестирование. Использованы property-based тесты QuickCheck для проверок неотрицательности продуктивности и зарплатных показателей.

## 1.3 Математическая модель

Для задачи сотрудника используется обобщенный балл продуктивности:

$$P_{task} = C \cdot K_c \cdot K_t \cdot K_r,$$

где $C$ — сложность задачи, $K_c$ — коэффициент типа/сложности, $K_t$ — коэффициент по времени выполнения, $K_r$ — коэффициент результата (success/partial/failed).

Интегральный KPI сотрудника за месяц:

$$KPI = \frac{\sum P_{task}}{N},$$

где $N$ — норматив продуктивности сотрудника.

Итоговая зарплата строится на базе фактически отработанных часов, ставки, премии, налогов, больничных и неоплачиваемых дней сотрудника. Введем:

$$H_{pay} = H_{fact} + D_{sick} \cdot H_{workDay} \cdot K_{sick} - D_{unpaid} \cdot H_{workDay},$$

где $H_{fact}$ — фактически отработанные часы, $D_{sick}$ — больничные дни, $D_{unpaid}$ — неоплачиваемые дни, $K_{sick}$ — коэффициент оплаты больничного.

Тогда базовая начисляемая часть:

$$S_{base} = R_{hour} \cdot H_{pay},$$

где $R_{hour}$ — часовая ставка. Далее применяются KPI-модификатор, бонусная шкала и суммарные удержания.

# 2 Примеры работы программы

Примеры работы программы и обработки ошибок представлены на рис. 1 – 7.

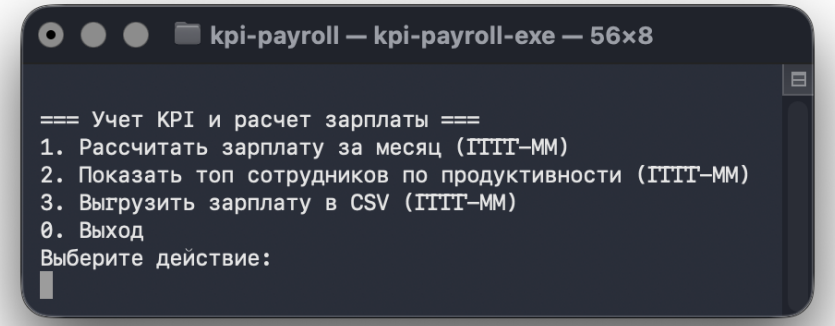



Рис. 1: Пользовательское меню

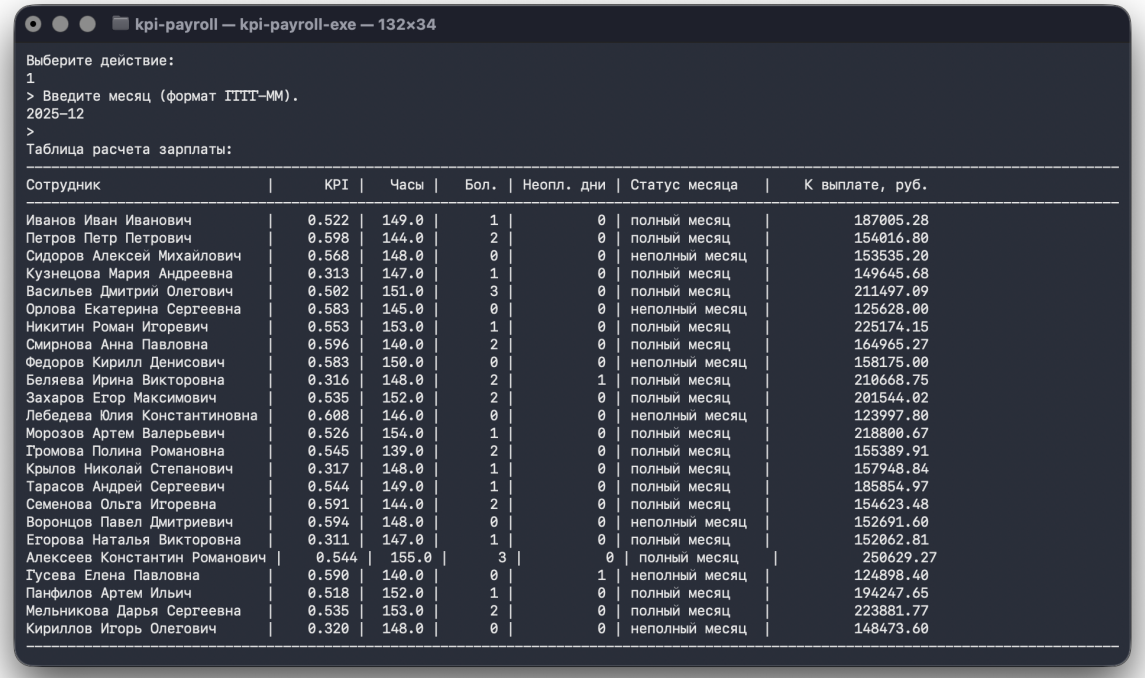



Рис. 2: Расчёт зарплаты

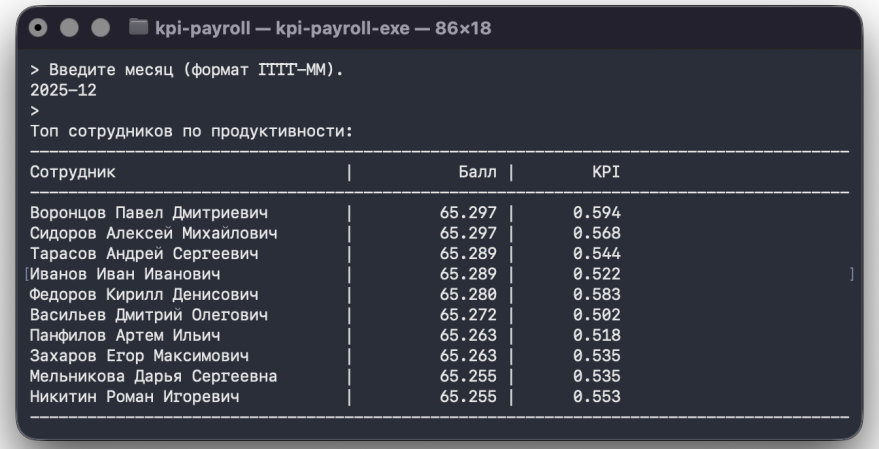



Рис. 3: Топ сотрудников по продуктивности

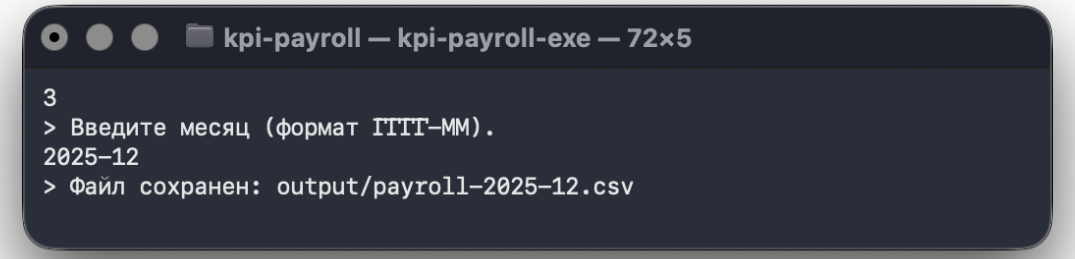



Рис. 4: Выгрузка зарплаты в CSV-файл

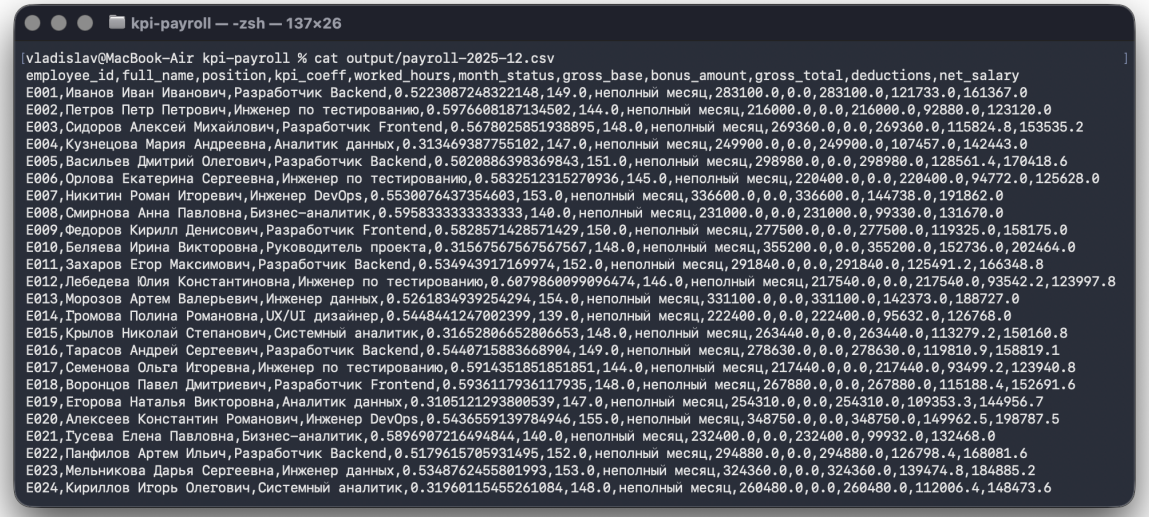



Рис. 5: Содержание выгруженного файла

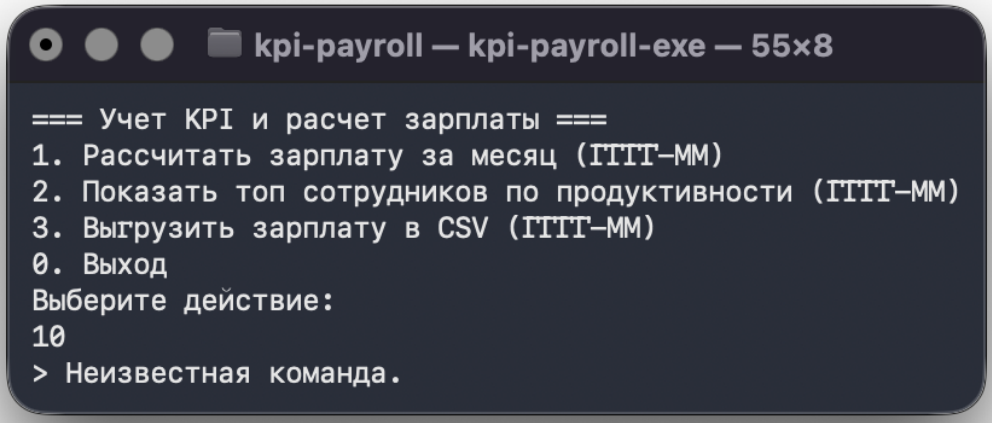



Рис. 6: Обработка некорректного ввода в пользовательском меню

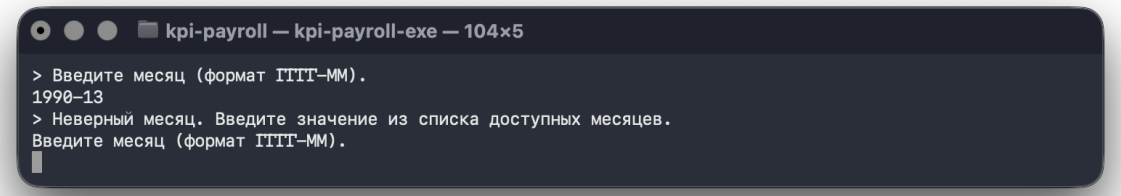



Рис. 7: Обработка некорректного ввода месяца

# 3 Руководство по эксплуатации для пользователя

## 3.1 Требования

- Установленный GHC и Stack.
- Наличие входных файлов в директории data/: employees.csv, tasks.csv, absences.csv, calendar.txt, config.json.

## 3.2 Порядок работы

1. Открыть терминал в каталоге kpi-payroll.
2. Выполнить stack run.
3. Выбрать действие в меню (см. рис. 1):
    - 1 — расчёт зарплаты (пример на рис. 2);
    - 2 — топ сотрудников по продуктивности (пример на рис. 3);
    - 3 — экспорт CSV (примеры на рис. 4 и 5).
4. Ввести месяц в формате ГГГГ-ММ (месяц должен присутствовать в загруженном файле tasks.csv).
5. При некорректном вводе программа обрабатывает ошибки (примеры на рис. 6 и 7).
6. Для выхода выбрать 0.

# Заключение

В ходе курсовой работы разработан программный комплекс для учета продуктивности сотрудников и расчета заработной платы на основе KPI. Реализованы загрузка и валидация входных данных, расчетные модули, консольный интерфейс, экспорт отчетов и логирование. Программа протестирована и демонстрирует корректную работу на расширенном наборе данных, приближенном к условиям реальной компании.

По результатам разработки выполнены ключевые требования ТЗ: модульность, обработка ошибок, наличие тестов, интерактивное меню, экспорт результатов и воспроизводимость расчетов.

# Выгрузка коммитов по этапам

| Этап | Дата | Коммит | Сообщение |
|---|---|---|---|
| ТЗ | 10.03 | 795b317 | first commit |
| ТЗ | 20.03 | 0aa01a4 | Добавлено техническое задание. |
| ТЗ | 26.03 | cda53f2 | Исправлено техническое задание. |
| ТЗ | 28.03 | 5f0a597 | Внесены изменения в техническое задание с учетом замечаний. |
| ТЗ | 28.03 | 2f7601b | Добавлена диаграмма Ганта |
| ТЗ | 03.04 | 7829727 | правки технического задания. Создана структура проета. |
| ТЗ | 03.04 | 5580cc7 | добавлен математический аппарат для расчета KPI и ЗП |
| ТЗ | 03.04 | 1c281d4 | добавлено описание типов данных |
| ТЗ | 08.04 | 82105fe | Исправлен титульный лист технического задания |
| ТЗ | 09.04 | cf9131a | добавлен отчет курсовой работы с разделом введение |
| Диаграмма | 03.04 | b5dd25a | Сделана диаграмма модулей |
| Структура | 08.04 | 8d9a14e | создание проекта в stack |
| Диаграмма | 10.04 | f9bbe74 | изменена диаграмма модулей |
| Диаграмма | 17.04 | 0e15b65 | правки диаграммы модулей |
| Диаграмма | 17.04 | 85280d2 | исправлена диаграмма модулей |
| Диаграмма | 17.04 | 138a2a1 | правки по диаграмме модулей |
| Диаграмма | 17.04 | 68f8cb4 | правки диаграммы модулей |
| Структура | 17.04 | 05110f0 | создана структура проекта |

| Этап | Дата | Коммит | Сообщение |
|---|---|---|---|
| Реализация | 24.04 | 70a54ee | сделана минимальная реализация проекта |
| Реализация | 01.05 | 960a0df | правки проекта |
| Реализация | 01.05 | 123c747 | добавлены данные(2025г), сделана валидация: корректный только 2025г. |
| Реализация | 01.05 | 80b9db2 | исправлен выброс на нач меню при неверном вводе. реализован вывод сотрудников, чьи значение продуктивности не нулевые |
| Реализация | 01.05 | 10f36b2 | добавлено больше данных, данные приближены к реальным |
| Тестирование | 06.05 | 30acecb | протестирована работа программы |
| Тестирование | 08.05 | f40f8221 | Добавлена обработка нерабочих дней, больничных, изменена формула расчета зп |

Фактические сроки выполнения работы представлены на диаграмме Ганта (рис. 8)

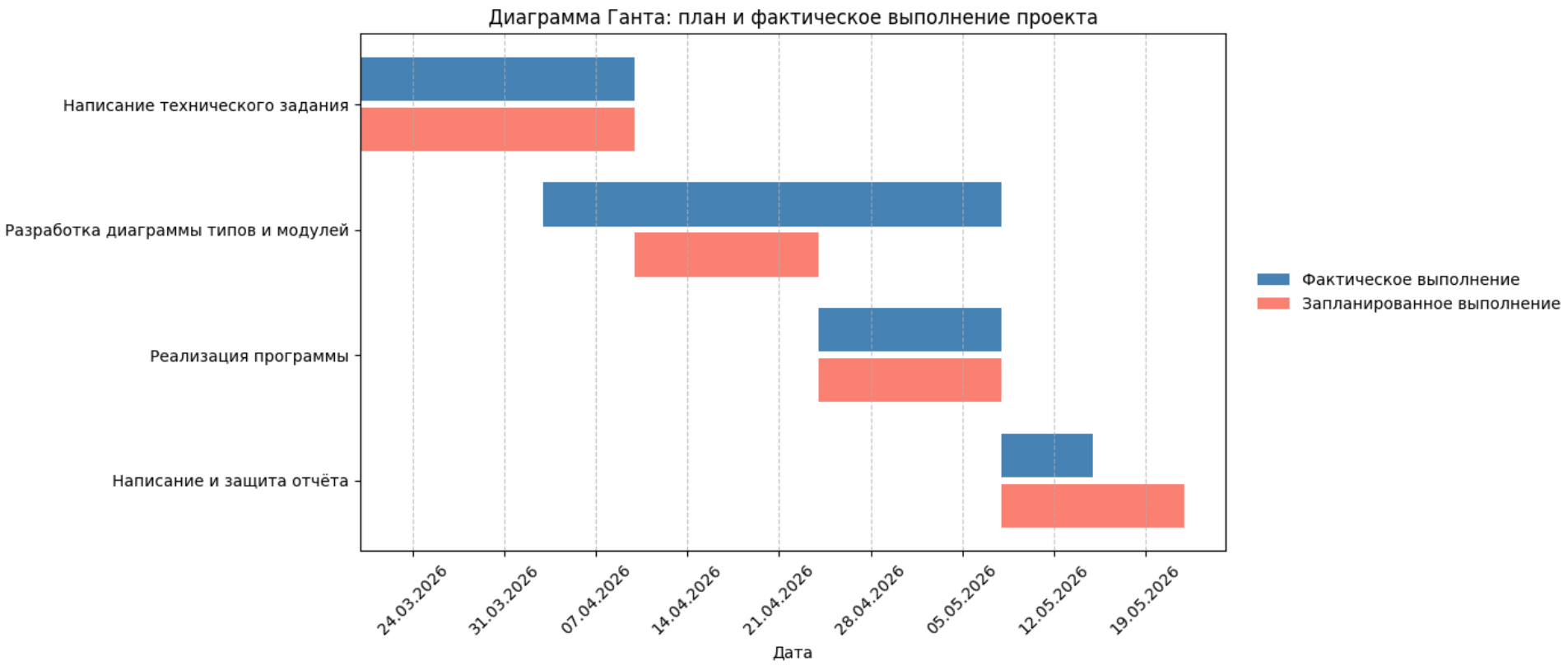



Рис. 8: Диаграмма Ганта

# Список литературы


[1] Курылева А. А. KPI как инструмент повышения эффективности системы управления персоналом // Экономика и социум. — 2020. — № 5.

[2] Черепанов Е. В. Принципы работы KPI и их применение в России // Молодой ученый. — 2017. — № 11. — С. 286–290.

[3] Методологические основы управления персоналом на основе ключевых показателей эффективности деятельности предприятия // Вестник Алтайской академии экономики и права. — 2019.

[4] Илюшников К. К. Инструменты построения модели оценки эффективности корпоративного обучения на основе системы KPI и детализированных метрик // Экономика труда. — 2018. — Т. 5, № 2. — С. 361–372.

[5] Баранова Э. С. Повышения эффективности системы мотивации персонала на основе внедрения программы KPI – Drive // Актуальные направления научных исследований XXI века: теория и практика. — 2024. — Т. 12, № 3. — С. 70–84. — DOI: 10.34220/2308-8877-2024-12-3-70-84.

[6] Bird R. Thinking Functionally with Haskell. — Cambridge University Press, 2014. — 352 с.

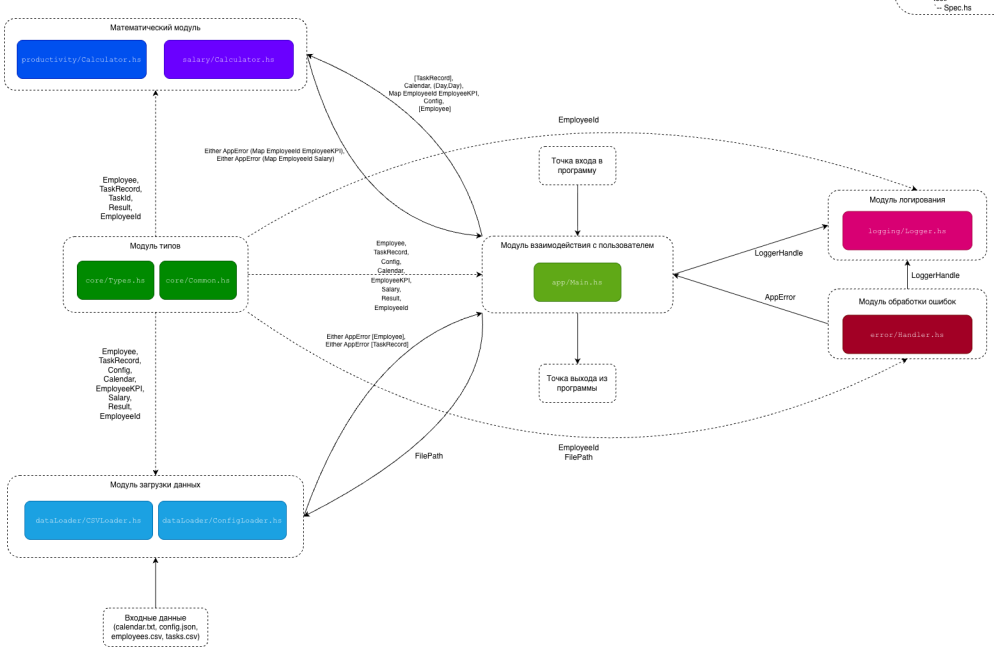
[7] Hutton G. Programming in Haskell. — 2nd ed. — Cambridge University Press, 2016. — 304 с.

[8] O'Sullivan B., Goerzen J., Stewart D. Real World Haskell. — O'Reilly Media, 2008. — 670 с.

[9] Lipovača M. Learn You a Haskell for Great Good! — No Starch Press, 2011. — 400 с.

Структура проекта

```
kpi-payroll/
├── CHANGELOG.md
├── LICENSE
├── README.md
├── Setup.hs
├── app/
│   └── Main.hs
├── data/
│   ├── calendar.txt
│   ├── config.json
│   ├── employees.csv
│   └── tasks.csv
├── kpi-payroll.cabal
├── logs/
│   └── app.log
├── package.yaml
├── src/
│   ├── Core/
│   │   ├── Common.hs
│   │   └── Types.hs
│   ├── DataLoader/
│   │   ├── CSVLoader.hs
│   │   └── ConfigLoader.hs
│   ├── Error/
│   │   ├── Handler.hs
│   │   └── Validation.hs
│   ├── Logging/
│   │   └── Logger.hs
│   ├── Productivity/
│   │   └── Calculator.hs
│   ├── Salary/
│   │   └── Calculator.hs
│   └── Utils/
│       ├── DateUtils.hs
│       └── Validation.hs
├── stack.yaml
└── test/
    └── Spec.hs
```




## Типы данных

Табельный номер сотрудника (EmployeeId)

```
newtype EmployeeId = EmployeeId Text
```

Идентификатор задачи (TaskId)

```
newtype TaskId = TaskId Text
```

Информация о сотруднике (Employee)

```
data Employee = Employee { empId :: EmployeeId, empName :: Text, empPosition :: Text, baseRate :: Decimal, productivityNorm ::Double }
```

Результаты выполнения задачи (Result)

```
data Result = Success | Partial | Failture
```

Запись о выполненной задаче (TaskRecord)

```
data TaskRecord = TaskRecord { taskEmployeeId :: EmployeeId, taskDate :: Day, taskId :: TaskId, taskComplexity :: Int, plannedHours :: Double, actualHours :: Double, taskResult :: Result }
```

Конфигурация расчета (Config)

```
data Config = Config { baseKPI :: Double, bonusScale :: [(Double, Double)],
taxRates :: TaxRates }
```

Ставки налогов (TaxRates)

```
data TaxRates = TaxRates { ndflRate :: Double, contributions :: Double }
```

Производственный календарь(Calendar)

```
data Calendar = Calendar { holidays :: Set Day, weekends :: Set DayOfWeek }
```

Промежуточный результат продуктивности задачи (TaskProductivity)

```
data TaskProductivity = TaskProductivity { rawScore :: Double, adjustedScore :: Double }
```

Интегральный KPI сотрудника за период (EmployeeKPI)

```
data EmployeeKPI = EmployeeKPI { totalProductivity :: Double, kpiCoefficient :: Double, periodDaysWorked :: Int }
```

Итоговая зарплата с налогами (Salry)

```
data Salary = Salary { baseSalary :: Decimal, productivityBonus :: Decimal, totalAccrued :: Decimal, taxDeduction :: Decimal, netSalary :: Decimal }
```

Обобщенный тип ошибок (AppError)

```
data AppError = FileNotFound FilePath | ParseError String | ValidationError String | DivisionByZero | EmployeeNotFound EmployeeId | InvalidPeriod String
```

Уровень важности логируемого сообщения (LogLevel)

```
data LogLevel = Info | Warning | Error
```

Дескриптор для работы с логгером(LoggerHandle)

```
data LoggerHandle = LoggerHandle { loggerSet :: LoggerSet, logFile :: FilePath }
```

Шкала бонусов(BonusScale)

```
type BonusScale = [(Double, Double)]
```

Начало и конец отчетного периода(Period)

```
type Period = (Day, Day)
```